

```

import { useState, useEffect, useRef } from "react";

// ——— CONSTANTS ———
const GIFTS = [
  "Plush Pepe", "Durov's Cap", "Jelly Bunny", "Magic Potion", "Loot Bag",
  "Vintage Cigar", "Eternal Candle", "Homemade Cake", "Sharp Tongue",
  "Spy Agaric", "Sakura Flower", "Spiced Wine", "Diamond Ring", "Evil Eye",
  "Frightful Egg", "Astral Shard", "Trapped Heart", "Skeleton Watch",
  "Voodoo Doll", "Hypno Lollipop", "Tama Gotchi", "Bunny Muffin",
  "Cookie Heart", "Witch Hat"
];

const MODELS = ["Common", "Rare", "Epic", "Legendary", "Mythical"];
const BACKDROPS = ["Space", "Nature", "Urban", "Abstract", "Fire", "Ice", "Gold", "Neon"];
const SYMBOLS = ["Moon", "Star", "Sun", "Heart", "Diamond", "Skull", "Crown", "Lightning"];

const DURATIONS = [
  { label: "1 Hour", value: "1h", stars: 10 },
  { label: "12 Hours", value: "12h", stars: 100 },
  { label: "24 Hours", value: "24h", stars: 200 },
  { label: "7 Days", value: "7d", stars: 1500 },
];

const FILTER_COSTS = { model: 5, backdrop: 5, symbol: 5, valueRange: 10 };
const LANGS = { EN: "English", RU: "Русский", ZH: "中文" };

const T = {
  EN: {
    terms_title: "Terms & Conditions",
    terms_body: `Welcome to GiftTrove – a Telegram gift scouting tool. By using t`,
    accept: "Accept & Continue",
    scout_tab: "Scout",
    radar_tab: "Radar",
    history_tab: "History",
    profile_tab: "Profile",
    select_gift: "Select Gift",
    select_model: "Model",
    select_backdrop: "Backdrop",
    select_symbol: "Symbol",
    value_range: "Value Range (Stars)",
    duration: "Scout Duration",
    launch: "Launch Scout",
    free_trial: "Free Trial – 10 min",
    free_trial_desc: "Try 1 filter, no stars needed",
  }
};

```

```

no_scouts: "No active scouts yet",
no_history: "No completed scouts",
stars_needed: "Get Stars with Hoton",
stars_desc: "Get 30% off via HotOnTG",
join_channel: "Join Channel",
community: "Community Chat",
support: "Support",
user_id: "User ID",
search_placeholder: "Search gifts...",
min: "Min", max: "Max",
optional: "optional",
active: "Active",
completed: "Completed",
view_details: "Details",
gift_required: "Please select a gift to scout",
duration_required: "Please select a duration",
},
RU: {
  terms_title: "Условия использования",
  terms_body: `Добро пожаловать в GiftTrove – инструмент поиска подарков в Tele
  accept: "Принять и продолжить",
  scout_tab: "Поиск",
  radar_tab: "Радар",
  history_tab: "История",
  profile_tab: "Профиль",
  select_gift: "Выбрать подарок",
  select_model: "Модель",
  select_backdrop: "Фон",
  select_symbol: "Символ",
  value_range: "Диапазон цен (Stars)",
  duration: "Длительность",
  launch: "Запустить",
  free_trial: "Пробный – 10 мин",
  free_trial_desc: "1 фильтр, без Stars",
  no_scouts: "Нет активных поисков",
  no_history: "Нет завершённых поисков",
  stars_needed: "Получить Stars с Hoton",
  stars_desc: "Скидка 30% через HotOnTG",
  join_channel: "Канал",
  community: "Чат сообщества",
  support: "Поддержка",
  user_id: "ID пользователя",
  search_placeholder: "Поиск подарков...",
  min: "Мин", max: "Макс",
  optional: "необязательно",
  active: "Активен",
  completed: "Завершён",

```

```

    view_details: "Подробнее",
    gift_required: "Выберите подарок",
    duration_required: "Выберите длительность",
  },
  ZH: {
    terms_title: "使用条款",
    terms_body: `欢迎使用 GiftTrove - Telegram 礼品侦测工具。使用本服务即表示您同意以下条i
    accept: "接受并继续",
    scout_tab: "侦测",
    radar_tab: "雷达",
    history_tab: "历史",
    profile_tab: "个人资料",
    select_gift: "选择礼品",
    select_model: "模型",
    select_backdrop: "背景",
    select_symbol: "符号",
    value_range: "价值范围 (Stars)",
    duration: "侦测时长",
    launch: "启动侦测",
    free_trial: "免费试用 - 10分钟",
    free_trial_desc: "1个筛选条件, 无需Stars",
    no_scouts: "暂无活跃侦测",
    no_history: "暂无历史记录",
    stars_needed: "通过Hoton获取Stars",
    stars_desc: "通过HotOnTG享受7折优惠",
    join_channel: "加入频道",
    community: "社区群组",
    support: "客服支持",
    user_id: "用户ID",
    search_placeholder: "搜索礼品...",
    min: "最低", max: "最高",
    optional: "可选",
    active: "活跃",
    completed: "已完成",
    view_details: "查看详情",
    gift_required: "请选择礼品",
    duration_required: "请选择侦测时长",
  }
};

const mockActiveScouts = [
  { id: 1, gift: "Plush Pepe", model: "Rare", backdrop: null, symbol: null, value
  { id: 2, gift: "Diamond Ring", model: null, backdrop: "Gold", symbol: "Crown",
];

const mockHistory = [
  { id: 10, gift: "Durov's Cap", model: "Legendary", backdrop: "Space", symbol: n
  { id: 11, gift: "Voodoo Doll", model: null, backdrop: null, symbol: null, value

```

```

];

function timeLeft(endsAt) {
  const diff = endsAt - Date.now();
  if (diff <= 0) return "Expired";
  const h = Math.floor(diff / 3600000);
  const m = Math.floor((diff % 3600000) / 60000);
  if (h > 0) return `${h}h ${m}m`;
  return `${m}m`;
}

function formatDate(ts) {
  return new Date(ts).toLocaleDateString("en-GB", { day: "2-digit", month: "short"
});

// SVG Icons
const IconSearch = () => (
  <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
    <circle cx="11" cy="11" r="8"/><path d="m21 21-4.35-4.35"/>
  </svg>
);

const IconRadar = () => (
  <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
    <path d="M19.07 4.93A10 10 0 0 0 0 6.99 3.34"/><path d="M4 6h.01"/><path d="M2.
  </svg>
);

const IconClock = () => (
  <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
    <circle cx="12" cy="12" r="10"/><polyline points="12 6 12 12 16 14"/>
  </svg>
);

const IconUser = () => (
  <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
    <path d="M20 21v-2a4 4 0 0 0 0-4-4H8a4 4 0 0 0 0-4 4v2"/><circle cx="12" cy="7" r
  </svg>
);

const IconChevronRight = () => (
  <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor"
    <polyline points="9 18 15 12 9 6"/>
  </svg>
);

const IconStar = () => (
  <svg width="14" height="14" viewBox="0 0 24 24" fill="currentColor" stroke="non
    <path d="M12 213.09 6.26L22 9.27l-5 4.87 1.18 6.88L12 17.77l-6.18 3.25L7 14.1
  </svg>
);

const IconGlobe = () => (

```

```

    <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor"
      <circle cx="12" cy="12" r="10"/><path d="M2 12h20"/><path d="M12 2a15.3 15.3
    </svg>
  );
  const IconZap = () => (
    <svg width="14" height="14" viewBox="0 0 24 24" fill="currentColor" stroke="none"
      <path d="M13 2L3 14h9l-1 8 10-12h-9l1-8z"/>
    </svg>
  );
  const IconCheck = () => (
    <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor"
      <polyline points="20 6 9 17 4 12"/>
    </svg>
  );
  const IconX = () => (
    <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
      <line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/>
    </svg>
  );
  const IconGift = () => (
    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor"
      <polyline points="20 12 20 22 4 22 4 12"/><rect x="2" y="7" width="20" height="
    </svg>
  );

  const styles = `
    @import url('https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600

  *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

  :root {
    --bg-deep: #0a0e1a;
    --bg-mid: #0d1220;
    --bg-card: rgba(255,255,255,0.04);
    --bg-card-hover: rgba(255,255,255,0.07);
    --tg-blue: #2AABEE;
    --tg-blue-dim: rgba(42,171,238,0.15);
    --tg-blue-glow: rgba(42,171,238,0.35);
    --tg-blue-border: rgba(42,171,238,0.25);
    --accent: #2AABEE;
    --accent-2: #1e96d4;
    --gold: #f5c842;
    --gold-dim: rgba(245,200,66,0.12);
    --text-primary: #e8edf5;
    --text-secondary: rgba(232,237,245,0.5);
    --text-muted: rgba(232,237,245,0.3);
    --border: rgba(255,255,255,0.08);
  }

```

```

--border-blue: rgba(42,171,238,0.2);
--radius-lg: 20px;
--radius-md: 14px;
--radius-sm: 10px;
--tab-h: 76px;
--safe-bottom: env(safe-area-inset-bottom, 0px);
--font: 'Sora', sans-serif;
--mono: 'JetBrains Mono', monospace;
}

html, body, #root { height: 100%; overflow: hidden; }

body {
  font-family: var(--font);
  background: var(--bg-deep);
  color: var(--text-primary);
  -webkit-font-smoothing: antialiased;
  overscroll-behavior: none;
}

/* BACKGROUND */
.bg-canvas {
  position: fixed; inset: 0; z-index: 0;
  background:
    radial-gradient(ellipse 70% 50% at 15% 5%, rgba(42,171,238,0.12) 0%, transp
    radial-gradient(ellipse 50% 40% at 85% 90%, rgba(30,150,212,0.08) 0%, trans
    radial-gradient(ellipse 80% 60% at 50% 50%, rgba(10,14,26,0.99) 0%, #070a14
}
.bg-noise {
  position: fixed; inset: 0; z-index: 0; opacity: 0.025;
  background-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 256 256' xmlns=
  background-size: 200px 200px;
}

/* LAYOUT */
.app { position: fixed; inset: 0; z-index: 1; display: flex; flex-direction: co

.content-area {
  flex: 1; overflow-y: auto; overflow-x: hidden;
  padding: 12px 16px calc(var(--tab-h) + var(--safe-bottom) + 20px);
  scroll-behavior: smooth;
}
.content-area::-webkit-scrollbar { display: none; }

/* GLASS CARD */
.glass {
  background: var(--bg-card);

```

```

border: 1px solid var(--border);
border-radius: var(--radius-lg);
backdrop-filter: blur(20px) saturate(1.5);
-webkit-backdrop-filter: blur(20px) saturate(1.5);
}

/* LOADING SCREEN */
.loading-screen {
  position: fixed; inset: 0; z-index: 999;
  display: flex; align-items: center; justify-content: center;
  background: var(--bg-deep);
  flex-direction: column; gap: 0;
  animation: fadeOut 0.5s ease 2s forwards;
}
.loading-img {
  width: 100%; height: 100%; object-fit: cover;
  position: absolute; inset: 0;
}
@keyframes fadeOut { from { opacity: 1; } to { opacity: 0; pointer-events: none } }

/* TERMS SCREEN */
.terms-overlay {
  position: fixed; inset: 0; z-index: 100;
  display: flex; align-items: flex-end; justify-content: center;
  background: rgba(7,10,20,0.9);
  backdrop-filter: blur(12px);
}
.terms-sheet {
  width: 100%; max-height: 90vh;
  border-radius: 28px 28px 0 0;
  padding: 8px 24px 32px;
  display: flex; flex-direction: column; gap: 16px;
  background: var(--bg-mid);
  border-top: 1px solid var(--border-blue);
  animation: slideUp 0.4s cubic-bezier(0.34,1.56,0.64,1);
}
.terms-drag { width: 40px; height: 4px; background: var(--border); border-radius:
.terms-header { display: flex; align-items: center; gap: 12px; }
.terms-logo-img { height: 32px; object-fit: contain; }
.terms-title { font-size: 18px; font-weight: 700; color: var(--text-primary); }
.terms-scroll { flex: 1; overflow-y: auto; font-size: 13px; line-height: 1.8; }
.terms-scroll::-webkit-scrollbar { display: none; }
.lang-row { display: flex; gap: 6px; justify-content: flex-end; flex-shrink: 0; }
.lang-btn {
  font-size: 11px; font-weight: 600; padding: 5px 12px; border-radius: 99px; cu
  background: var(--bg-card); color: var(--text-secondary); border: 1px solid v
  font-family: var(--font);

```

```

}
.lang-btn.active { background: var(--tg-blue-dim); color: var(--tg-blue); borde

/* HEADER */
.header {
  display: flex; align-items: center; justify-content: space-between;
  padding: 14px 16px 6px;
  flex-shrink: 0;
}
.logo { display: flex; align-items: center; }
.logo-img { height: 30px; object-fit: contain; }
.header-right { display: flex; align-items: center; gap: 8px; }
.lang-pill {
  display: flex; align-items: center; gap: 5px;
  font-size: 11px; font-weight: 600; padding: 5px 10px; border-radius: 99px;
  background: var(--bg-card); border: 1px solid var(--border);
  color: var(--text-secondary); cursor: pointer; font-family: var(--font);
}

/* TAB BAR - Hoton style pill selector */
.tab-bar-wrap {
  position: fixed; bottom: 0; left: 0; right: 0; z-index: 60;
  padding-bottom: var(--safe-bottom);
  background: rgba(10,14,26,0.92);
  border-top: 1px solid var(--border);
  backdrop-filter: blur(24px);
  -webkit-backdrop-filter: blur(24px);
}
.tab-bar {
  display: flex; align-items: center; justify-content: space-around;
  padding: 10px 12px 8px;
  height: var(--tab-h);
  position: relative;
}
.tab-item {
  flex: 1; display: flex; flex-direction: column; align-items: center; gap: 4px;
  cursor: pointer; padding: 6px 4px; border-radius: 14px; transition: all 0.25s;
  border: none; background: none; color: var(--text-muted); position: relative;
  font-family: var(--font);
}
.tab-item.active { color: var(--tg-blue); }
.tab-icon-pill {
  width: 44px; height: 30px; border-radius: 10px;
  display: flex; align-items: center; justify-content: center;
  transition: all 0.3s cubic-bezier(0.34,1.56,0.64,1);
}
.tab-item.active .tab-icon-pill {

```

```

    background: var(--tg-blue-dim);
    box-shadow: 0 0 16px var(--tg-blue-glow);
    transform: translateY(-2px);
}
.tab-label { font-size: 10px; font-weight: 600; letter-spacing: 0.2px; transiti

/* SECTION TITLE */
.section-title { font-size: 11px; font-weight: 700; letter-spacing: 1.2px; text

/* SCOUT TAB */
.scout-card { padding: 16px; display: flex; flex-direction: column; gap: 14px;
.cost-bar {
    display: flex; align-items: center; justify-content: space-between;
    padding: 12px 16px; border-radius: var(--radius-md);
    background: var(--tg-blue-dim); border: 1px solid var(--tg-blue-border);
}
.cost-label { font-size: 12px; color: var(--text-secondary); font-weight: 500;
.cost-value { font-family: var(--mono); font-size: 22px; font-weight: 600; colo
.cost-star-icon { color: var(--gold); display: flex; }

/* Gift search */
.gift-search-wrap { position: relative; }
.gift-search-input {
    width: 100%; padding: 12px 14px 12px 40px;
    background: var(--bg-card); border: 1px solid var(--border);
    border-radius: var(--radius-md); color: var(--text-primary); font-family: var
    outline: none; transition: border-color 0.2s;
}
.gift-search-input:focus { border-color: var(--tg-blue-border); background: rgb
.gift-search-input::placeholder { color: var(--text-muted); }
.gift-search-icon { position: absolute; left: 13px; top: 50%; transform: transl
.gift-dropdown {
    position: absolute; left: 0; right: 0; top: calc(100% + 6px); z-index: 20;
    max-height: 200px; overflow-y: auto; border-radius: var(--radius-md);
    border: 1px solid var(--border);
    background: rgba(13,18,32,0.98); backdrop-filter: blur(24px);
    box-shadow: 0 8px 32px rgba(0,0,0,0.4);
}
.gift-dropdown::-webkit-scrollbar { display: none; }
.gift-option {
    padding: 11px 14px; font-size: 13.5px; cursor: pointer; transition: backgroun
    display: flex; align-items: center; gap: 10px; color: var(--text-secondary);
}
.gift-option:hover, .gift-option.selected { background: var(--tg-blue-dim); col
.gift-option-dot { width: 7px; height: 7px; border-radius: 50%; background: var
.gift-option.selected .gift-option-dot { background: var(--tg-blue); }
.gift-check { margin-left: auto; color: var(--tg-blue); display: flex; }

```

```

/* Filter chips */
.filter-label { font-size: 11.5px; font-weight: 600; color: var(--text-secondary); }
.filter-label-tags { display: flex; gap: 4px; align-items: center; }
.optional-tag { font-size: 10px; color: var(--text-muted); font-weight: 500; }
.cost-tag { font-size: 10px; color: var(--tg-blue); font-weight: 700; background-color: var(--bg-card); }
.chips { display: flex; flex-wrap: wrap; gap: 7px; }
.chip {
  padding: 7px 14px; border-radius: 99px; font-size: 12.5px; font-weight: 600;
  cursor: pointer; transition: all 0.2s; border: 1px solid var(--border);
  background: var(--bg-card); color: var(--text-secondary);
  user-select: none; -webkit-user-select: none; font-family: var(--font);
}
.chip:active { transform: scale(0.95); }
.chip.selected { background: var(--tg-blue-dim); border-color: var(--tg-blue-border); }

/* Value range */
.range-row { display: flex; gap: 8px; }
.range-input {
  flex: 1; padding: 11px 12px; border-radius: var(--radius-sm); font-size: 14px;
  background: var(--bg-card); border: 1px solid var(--border);
  color: var(--text-primary); outline: none; transition: border-color 0.2s;
}
.range-input:focus { border-color: var(--tg-blue-border); }
.range-input::placeholder { color: var(--text-muted); font-size: 12px; }

/* Duration */
.duration-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 8px; }
.duration-card {
  padding: 13px; border-radius: var(--radius-md); cursor: pointer; transition:
  border 1px solid var(--border); background: var(--bg-card);
  display: flex; flex-direction: column; gap: 3px;
  user-select: none; -webkit-user-select: none;
}
.duration-card:active { transform: scale(0.97); }
.duration-card.selected { border-color: var(--tg-blue-border); background: var(--tg-blue-dim); }
.duration-name { font-size: 13.5px; font-weight: 700; color: var(--text-primary); }
.duration-stars { font-size: 11.5px; color: var(--gold); font-family: var(--monospace); }

/* Launch button */
.launch-btn {
  width: 100%; padding: 16px; border-radius: var(--radius-md); border: none;
  font-family: var(--font); font-size: 15px; font-weight: 700;
  background: linear-gradient(135deg, var(--tg-blue), var(--accent-2));
  color: white; letter-spacing: 0.3px;
  box-shadow: 0 4px 24px var(--tg-blue-glow);
  transition: all 0.2s; display: flex; align-items: center; justify-content: center;
}

```

```

}
.launch-btn:active { transform: scale(0.98); box-shadow: 0 2px 12px var(--tg-bl
.launch-btn:disabled { opacity: 0.35; cursor: not-allowed; transform: none; box
.launch-note { text-align: center; font-size: 11px; color: var(--text-muted); m

/* FREE TRIAL BUBBLE */
.trial-bubble {
  position: fixed; bottom: calc(var(--tab-h) + var(--safe-bottom) + 16px); righ
  z-index: 50; cursor: pointer;
  animation: float 3s ease-in-out infinite;
}
.trial-bubble-inner {
  background: linear-gradient(135deg, #1e3a5f, #154a7a);
  border: 1px solid var(--tg-blue-border);
  border-radius: 50px; padding: 8px 14px;
  display: flex; align-items: center; gap: 6px;
  box-shadow: 0 4px 20px var(--tg-blue-glow);
  font-size: 12px; font-weight: 700; color: var(--tg-blue);
}

/* RADAR TAB */
.scout-item {
  padding: 15px; border-radius: var(--radius-md); display: flex; flex-direction
  margin-bottom: 10px;
}
.scout-item-header { display: flex; align-items: center; justify-content: space
.scout-item-name { font-size: 14.5px; font-weight: 700; color: var(--text-prima
.badge { font-size: 10px; font-weight: 700; padding: 3px 9px; border-radius: 99
.badge-active { background: rgba(42,171,238,0.15); color: var(--tg-blue); borde
.badge-done { background: var(--bg-card); color: var(--text-muted); border: 1px
.scout-item-meta { display: flex; flex-wrap: wrap; gap: 5px; }
.meta-chip { font-size: 11px; padding: 3px 9px; border-radius: 99px; background
.scout-item-footer { display: flex; align-items: center; justify-content: space
.time-left { font-family: var(--mono); font-size: 12px; color: var(--tg-blue);
.results-count { font-size: 11.5px; color: var(--text-muted); margin-top: 2px;
.detail-btn {
  font-size: 12px; font-weight: 600; padding: 6px 14px; border-radius: 8px; cur
  background: var(--tg-blue-dim); border: 1px solid var(--tg-blue-border); colo
  font-family: var(--font); transition: all 0.2s;
}
.detail-btn:active { transform: scale(0.96); }
.pulse-dot {
  width: 7px; height: 7px; border-radius: 50%; background: var(--tg-blue);
  animation: pulseAnim 2s ease-in-out infinite; flex-shrink: 0;
  box-shadow: 0 0 8px var(--tg-blue);
}
.empty-state { padding: 48px 24px; text-align: center; display: flex; flex-dire

```

```

.empty-icon-wrap { width: 64px; height: 64px; border-radius: 20px; background:
.empty-text { font-size: 14px; color: var(--text-secondary); }

/* HISTORY TAB */
.history-item { padding: 15px; border-radius: var(--radius-md); margin-bottom:
.history-header { display: flex; justify-content: space-between; align-items: f
.history-name { font-size: 14px; font-weight: 700; color: var(--text-primary);
.history-date { font-size: 11px; color: var(--text-muted); }
.history-filters { display: flex; flex-wrap: wrap; gap: 5px; margin-bottom: 10p
.history-footer { display: flex; justify-content: space-between; align-items: c
.stars-cost { font-family: var(--mono); font-size: 12.5px; color: var(--gold);
.results-num { font-size: 12px; color: var(--text-muted); }

/* PROFILE TAB */
.profile-hero {
  padding: 28px 20px 24px; text-align: center;
  display: flex; flex-direction: column; align-items: center; gap: 12px;
  border-radius: var(--radius-lg); margin-bottom: 12px;
  background: var(--bg-card); border: 1px solid var(--border);
}
.avatar-ring {
  width: 80px; height: 80px; border-radius: 50%;
  background: linear-gradient(135deg, var(--tg-blue), var(--accent-2));
  padding: 3px;
  box-shadow: 0 0 20px var(--tg-blue-glow), 0 0 40px rgba(42,171,238,0.15);
  animation: avatarGlow 3s ease-in-out infinite;
}
.avatar-inner {
  width: 100%; height: 100%; border-radius: 50%;
  background: var(--bg-mid);
  display: flex; align-items: center; justify-content: center;
  font-size: 30px;
}
@keyframes avatarGlow {
  0%, 100% { box-shadow: 0 0 20px var(--tg-blue-glow), 0 0 40px rgba(42,171,238
  50% { box-shadow: 0 0 30px var(--tg-blue-glow), 0 0 60px rgba(42,171,238,0.25
}
.profile-name { font-size: 20px; font-weight: 800; color: var(--text-primary);
.profile-id { font-family: var(--mono); font-size: 11.5px; color: var(--text-mu

/* Stars promo - Billy Westor image */
.stars-promo-wrap {
  display: block; margin-bottom: 12px; border-radius: var(--radius-lg);
  overflow: hidden; border: 1px solid rgba(245,200,66,0.2);
  text-decoration: none; cursor: pointer;
  transition: transform 0.2s, box-shadow 0.2s;
}

```

```

.stars-promo-wrap:active { transform: scale(0.98); }
.stars-promo-img { width: 100%; display: block; }

/* Sticker cards */
.sticker-links { display: flex; gap: 10px; margin-bottom: 12px; }
.sticker-card {
  flex: 1; padding: 16px 12px; border-radius: var(--radius-lg);
  background: var(--bg-card); border: 1px solid var(--border);
  display: flex; flex-direction: column; align-items: center; gap: 10px;
  text-decoration: none; color: var(--text-primary); cursor: pointer;
  transition: all 0.2s; overflow: hidden;
}
.sticker-card:active { transform: scale(0.97); background: var(--bg-card-hover) }
.sticker-img { width: 60px; height: 60px; object-fit: contain; }
.sticker-label { font-size: 12px; font-weight: 700; color: var(--text-secondary)

/* Profile links */
.profile-links { border-radius: var(--radius-lg); overflow: hidden; background:
.profile-link {
  padding: 16px; display: flex; align-items: center; gap: 14px;
  cursor: pointer; text-decoration: none; color: var(--text-primary);
  transition: background 0.2s; border: none; background: none; width: 100%; fon
}
.profile-link:active { background: var(--bg-card-hover); }
.link-icon-wrap {
  width: 38px; height: 38px; border-radius: 12px;
  background: var(--tg-blue-dim); border: 1px solid var(--tg-blue-border);
  display: flex; align-items: center; justify-content: center;
  overflow: hidden; flex-shrink: 0;
}
.link-icon-img { width: 38px; height: 38px; object-fit: cover; border-radius: 1
.link-text { font-size: 14px; font-weight: 600; flex: 1; text-align: left; }
.link-arrow { color: var(--text-muted); display: flex; }
.divider { height: 1px; background: var(--border); margin: 0 16px; }

/* MODAL */
.modal-overlay {
  position: fixed; inset: 0; z-index: 80;
  background: rgba(7,10,20,0.85); backdrop-filter: blur(12px);
  display: flex; align-items: flex-end;
  animation: fadeIn 0.2s ease;
}
.modal-sheet {
  width: 100%; max-height: 80vh; border-radius: 28px 28px 0 0;
  padding: 8px 20px 32px; overflow-y: auto;
  background: var(--bg-mid);
  border-top: 1px solid var(--border-blue);

```

```

        animation: slideUp 0.3s cubic-bezier(0.34,1.56,0.64,1);
    }
    .modal-sheet::-webkit-scrollbar { display: none; }
    .modal-handle { width: 40px; height: 4px; background: var(--border); border-rad
    .modal-title { font-size: 16px; font-weight: 800; color: var(--text-primary); m
    .modal-row { display: flex; justify-content: space-between; align-items: center
    .modal-label { font-size: 12px; color: var(--text-muted); }
    .modal-value { font-size: 13px; font-weight: 600; color: var(--text-primary); }
    .close-btn {
        width: 100%; padding: 14px; border-radius: var(--radius-md); border: 1px soli
        background: var(--bg-card); color: var(--text-secondary); font-family: var(--
        font-size: 14px; font-weight: 600; cursor: pointer; margin-top: 16px; transit
    }
    .close-btn:active { background: var(--bg-card-hover); }

    /* TOAST */
    .toast {
        position: fixed; top: 16px; left: 50%; transform: translateX(-50%); z-index:
        padding: 10px 20px; border-radius: 99px; font-size: 13px; font-weight: 600;
        background: var(--bg-mid); border: 1px solid var(--tg-blue-border); color: va
        box-shadow: 0 4px 24px rgba(0,0,0,0.5);
        animation: toastIn 0.3s ease, toastOut 0.3s ease 2.7s forwards;
        white-space: nowrap; font-family: var(--font);
    }

    /* ANIMATIONS */
    @keyframes slideUp { from { transform: translateY(100%); } to { transform: tran
    @keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }
    @keyframes float { 0%,100% { transform: translateY(0); } 50% { transform: trans
    @keyframes pulseAnim { 0%,100% { opacity: 1; transform: scale(1); } 50% { opaci
    @keyframes toastIn { from { opacity: 0; transform: translateX(-50%) translateY(
    @keyframes toastOut { from { opacity: 1; } to { opacity: 0; } }
    @keyframes tabIn { from { opacity: 0; transform: translateY(8px); } to { opacit

    .tab-content { animation: tabIn 0.25s ease; }
`;

export default function App() {
    const [loading, setLoading] = useState(true);
    const [termsAccepted, setTermsAccepted] = useState(() => {
        try { return localStorage.getItem("gt_terms") === "1"; } catch { return false
    });
    const [lang, setLang] = useState(() => {
        try { return localStorage.getItem("gt_lang") || "EN"; } catch { return "EN";
    });
    const [tab, setTab] = useState("scout");
    const [showTrialBubble, setShowTrialBubble] = useState(true);

```

```

const [toast, setToast] = useState(null);
const [detailScout, setDetailScout] = useState(null);
const [showLangPicker, setShowLangPicker] = useState(false);

const [giftSearch, setGiftSearch] = useState("");
const [giftDropdown, setGiftDropdown] = useState(false);
const [selectedGift, setSelectedGift] = useState(null);
const [selectedModel, setSelectedModel] = useState(null);
const [selectedBackdrop, setSelectedBackdrop] = useState(null);
const [selectedSymbol, setSelectedSymbol] = useState(null);
const [minVal, setMinVal] = useState("");
const [maxVal, setMaxVal] = useState("");
const [selectedDuration, setSelectedDuration] = useState(null);

const t = T[lang];
const tgUser = typeof window !== 'undefined' ? window.Telegram?.WebApp?.initDat

useEffect(() => {
  const timer = setTimeout(() => setLoading(false), 2500);
  return () => clearTimeout(timer);
}, []);

const cost = (selectedGift ? 5 : 0)
  + (selectedModel ? FILTER_COSTS.model : 0)
  + (selectedBackdrop ? FILTER_COSTS.backdrop : 0)
  + (selectedSymbol ? FILTER_COSTS.symbol : 0)
  + ((minVal || maxVal) ? FILTER_COSTS.valueRange : 0)
  + (selectedDuration ? selectedDuration.stars : 0);

const filteredGifts = GIFTS.filter(g => g.toLowerCase().includes(giftSearch.toL

function showToast(msg) {
  setToast(msg);
  setTimeout(() => setToast(null), 3000);
}

function acceptTerms() {
  try { localStorage.setItem("gt_terms", "1"); } catch {}
  setTermsAccepted(true);
}

function changeLang(l) {
  setLang(l);
  try { localStorage.setItem("gt_lang", l); } catch {}
  setShowLangPicker(false);
}

function handleLaunch() {
  if (!selectedGift) { showToast(t.gift_required); return; }
  if (!selectedDuration) { showToast(t.duration_required); return; }

```

```

        showToast(`Scout launched for ${selectedGift}! (${cost} Stars)`);
        setSelectedGift(null); setGiftSearch(""); setSelectedModel(null);
        setSelectedBackdrop(null); setSelectedSymbol(null);
        setMinVal(""); setMaxVal(""); setSelectedDuration(null);
        setTab("radar");
    }
    function handleFreeTrial() {
        showToast("Free 10-min trial started!");
        setShowTrialBubble(false);
        setTab("radar");
    }

    const tabs = [
        { id: "scout", icon: <IconSearch />, label: t.scout_tab },
        { id: "radar", icon: <IconRadar />, label: t.radar_tab },
        { id: "history", icon: <IconClock />, label: t.history_tab },
        { id: "profile", icon: <IconUser />, label: t.profile_tab },
    ];

    // Telegram sticker file IDs for display
    const joinChannelStickerId = "CAACAgQAAxkBAAFLh-JqIdGYK3WHTFUedW4O9Fip8JoZNwACj
    const communityChatStickerId = "CAACAgQAAxkBAAFLh-NqIdGYFjdwXKAGdKpUEUVEh9xrBQA

    // We'll use the Telegram CDN thumbnail URLs for the stickers
    const joinChannelThumbId = "AAMCBAADGQEAAUuH4moh0ZgrdYdMVR5lbg70Uinwmhk3AAKMDwA
    const communityThumbId = "AAMCBAADGQEAAUuH42oh0ZgWN3BcoAZ0qlQRRUSH3GsFAAIOFAACE

    return (
        <>
        <style>{styles}</style>
        <div className="bg-canvas" />
        <div className="bg-noise" />

        {/* LOADING SCREEN */}
        {loading && (
            <div className="loading-screen">
                
            </div>
        )}

        {/* TERMS */}
        {!loading && !termsAccepted && (
            <div className="terms-overlay">

```

```

<div className="terms-sheet">
  <div className="terms-drag" />
  <div className="lang-row">
    {Object.keys(LANGS).map(l => (
      <button key={l} className={`lang-btn ${lang === l ? "active" : ""}
    ))}
  </div>
  <div className="terms-header">
    {t.terms_title}</div>
  </div>
  <div className="terms-scroll">{t.terms_body}</div>
  <button className="launch-btn" onClick={acceptTerms}>{t.accept}</butt
</div>
</div>
)}

```

```

{/* TOAST */}
{toast && <div className="toast">{toast}</div>}

```

```

{/* DETAIL MODAL */}
{detailScout && (
  <div className="modal-overlay" onClick={() => setDetailScout(null)}>
    <div className="modal-sheet" onClick={e => e.stopPropagation()}>
      <div className="modal-handle" />
      <div className="modal-title">
        <IconGift /> {detailScout.gift}
      </div>
      {detailScout.model && <div className="modal-row"><span className="mod
      {detailScout.backdrop && <div className="modal-row"><span className="
      {detailScout.symbol && <div className="modal-row"><span className="mo
      {detailScout.valueRange && <div className="modal-row"><span className
      <div className="modal-row"><span className="modal-label">{t.duration}
      <div className="modal-row"><span className="modal-label">Stars Spent<
      <div className="modal-row"><span className="modal-label">Results Foun
      {detailScout.endsAt && <div className="modal-row"><span className="mo
      {detailScout.completedAt && <div className="modal-row"><span classNam
      <button className="close-btn" onClick={() => setDetailScout(null)}>Cl
    </div>
  </div>
)}

```

```

{/* LANG PICKER */}
{showLangPicker && (
  <div className="modal-overlay" onClick={() => setShowLangPicker(false)}>
    <div className="modal-sheet" onClick={e => e.stopPropagation()}>
      <div className="modal-handle" />

```

```

<div className="modal-title"><IconGlobe /> Language</div>
{Object.entries(LANGS).map(([k, v]) => (
  <div key={k}
    onClick={() => changeLang(k)}
    style={{
      padding: "14px 16px", borderRadius: 12, marginBottom: 6,
      display: "flex", alignItems: "center", justifyContent: "space-b
      background: lang === k ? "var(--tg-blue-dim)" : "var(--bg-card)
      border: `1px solid ${lang === k ? "var(--tg-blue-border)" : "va
      cursor: "pointer", transition: "all 0.2s"
    }}>
    <span style={{ fontSize: 14, fontWeight: 600 }}>{v}</span>
    {lang === k && <span style={{ color: "var(--tg-blue)", display: "
  </div>
)}}
</div>
</div>
)}

{!loading && (
  <div className="app">
    {/* HEADER */}
    <div className="header">
      <div className="logo">
        
      <div className="header-right">
        <button className="lang-pill" onClick={() => setShowLangPicker(true
          <IconGlobe /> {lang}
        </button>
      </div>
    </div>

    {/* FREE TRIAL BUBBLE */}
    {showTrialBubble && tab === "scout" && (
      <div className="trial-bubble" onClick={handleFreeTrial}>
        <div className="trial-bubble-inner">
          <IconZap />
          {t.free_trial}
        </div>
      </div>
    )}

    {/* CONTENT */}
    <div className="content-area">

      {/* — SCOUT TAB — */}

```

```

{tab === "scout" && (
  <div className="tab-content">
    <div className="section-title">{t.scout_tab}</div>
    <div className="glass scout-card">
      {/* Cost bar */}
      <div className="cost-bar">
        <span className="cost-label">Total Cost</span>
        <span className="cost-value">
          <span className="cost-star-icon"><IconStar /></span>
          {cost}
        </span>
      </div>

      {/* Gift selector */}
      <div>
        <div className="filter-label">
          {t.select_gift}
          <span className="cost-tag"><IconStar /> 5</span>
        </div>
        <div className="gift-search-wrap">
          <span className="gift-search-icon"><IconSearch /></span>
          <input
            className="gift-search-input"
            placeholder={t.search_placeholder}
            value={selectedGift || giftSearch}
            onChange={e => { setGiftSearch(e.target.value); setSelect
              onFocus={() => setGiftDropdown(true)}
            }
          />
          {giftDropdown && filteredGifts.length > 0 && (
            <div className="gift-dropdown">
              {filteredGifts.map(g => (
                <div key={g} className={`gift-option ${selectedGift =
                  onClick={() => { setSelectedGift(g); setGiftSearch(
                    <span className="gift-option-dot" />
                    {g}
                    {selectedGift === g && <span className="gift-check"
                  </div>
                )}}
              </div>
            )}}
          </div>
        </div>
      </div>
    </div>

    {/* Model */}
    <div>
      <div className="filter-label">
        {t.select_model}

```

```

        <div className="filter-label-tags">
          <span className="optional-tag">{t.optional}</span>
          <span className="cost-tag"><IconStar /> 5</span>
        </div>
      </div>
    <div className="chips">
      {MODELS.map(m => (
        <div key={m} className={`chip ${selectedModel === m ? "se
          onClick={() => setSelectedModel(selectedModel === m ? n
        )})}
      </div>
    </div>

    {/* Backdrop */}
    <div>
      <div className="filter-label">
        {t.select_backdrop}
        <div className="filter-label-tags">
          <span className="optional-tag">{t.optional}</span>
          <span className="cost-tag"><IconStar /> 5</span>
        </div>
      </div>
      <div className="chips">
        {BACKDROPS.map(b => (
          <div key={b} className={`chip ${selectedBackdrop === b ?
            onClick={() => setSelectedBackdrop(selectedBackdrop ===
          )})}
        </div>
      </div>
    </div>

    {/* Symbol */}
    <div>
      <div className="filter-label">
        {t.select_symbol}
        <div className="filter-label-tags">
          <span className="optional-tag">{t.optional}</span>
          <span className="cost-tag"><IconStar /> 5</span>
        </div>
      </div>
      <div className="chips">
        {SYMBOLS.map(s => (
          <div key={s} className={`chip ${selectedSymbol === s ? "s
            onClick={() => setSelectedSymbol(selectedSymbol === s ?
          )})}
        </div>
      </div>
    </div>

```

```

    { /* Value range */ }
    <div>
      <div className="filter-label">
        {t.value_range}
        <div className="filter-label-tags">
          <span className="optional-tag">{t.optional}</span>
          <span className="cost-tag"><IconStar /> 10</span>
        </div>
      </div>
      <div className="range-row">
        <input className="range-input" placeholder={t.min} type="nu
        <input className="range-input" placeholder={t.max} type="nu
      </div>
    </div>

    { /* Duration */ }
    <div>
      <div className="filter-label">{t.duration}</div>
      <div className="duration-grid">
        {DURATIONS.map(d => (
          <div key={d.value} className={`duration-card ${selectedDu
            onClick={() => setSelectedDuration(selectedDuration?.va
            <div className="duration-name">{d.label}</div>
            <div className="duration-stars"><IconStar /> {d.stars.t
          </div>
        ))}
      </div>
    </div>

    { /* Launch */ }
    <button className="launch-btn" onClick={handleLaunch} disabled=
      {t.launch} {cost > 0 ? `· ${cost} Stars` : ""}
    </button>
    <div className="launch-note">Stars are non-refundable after lau
  </div>
</div>
)}

{ /* — RADAR TAB — */ }
{tab === "radar" && (
  <div className="tab-content">
    <div className="section-title">{t.active}</div>
    {mockActiveScouts.length === 0 ? (
      <div className="glass empty-state">
        <div className="empty-icon-wrap"><IconRadar /></div>
        <div className="empty-text">{t.no_scouts}</div>
      </div>
    )
  }
}

```

```

) : mockActiveScouts.map(s => (
  <div key={s.id} className="glass scout-item">
    <div className="scout-item-header">
      <div style={{display:"flex",alignItems:"center",gap:8}}>
        <div className="pulse-dot" />
        <span className="scout-item-name">{s.gift}</span>
      </div>
      <span className="badge badge-active">{t.active}</span>
    </div>
    <div className="scout-item-meta">
      {s.model && <span className="meta-chip">Model: {s.model}</s
      {s.backdrop && <span className="meta-chip">Backdrop: {s.bac
      {s.symbol && <span className="meta-chip">Symbol: {s.symbol}
      {s.valueRange && <span className="meta-chip">{s.valueRange[
      <span className="meta-chip">{s.duration}</span>
    </div>
    <div className="scout-item-footer">
      <div>
        <div className="time-left">{timeLeft(s.endsAt)} left</div>
        <div className="results-count">{s.results} results found<
      </div>
      <button className="detail-btn" onClick={() => setDetailScou
    </div>
  </div>
)}}
</div>
))
))

{/* — HISTORY TAB — */}
{tab === "history" && (
  <div className="tab-content">
    <div className="section-title">{t.history_tab}</div>
    {mockHistory.length === 0 ? (
      <div className="glass empty-state">
        <div className="empty-icon-wrap"><IconClock /></div>
        <div className="empty-text">{t.no_history}</div>
      </div>
    ) : mockHistory.map(s => (
      <div key={s.id} className="glass history-item">
        <div className="history-header">
          <div className="history-name">{s.gift}</div>
          <div className="history-date">{formatDate(s.completedAt)}</
        </div>
        <div className="history-filters">
          {s.model && <span className="meta-chip">Model: {s.model}</s
          {s.backdrop && <span className="meta-chip">Backdrop: {s.bac
          {s.symbol && <span className="meta-chip">Symbol: {s.symbol}

```

```

        {s.valueRange && <span className="meta-chip">{s.valueRange[
        <span className="meta-chip">{s.duration}</span>
    </div>
    <div className="history-footer">
        <span className="stars-cost"><IconStar /> {s.starsSpent} St
        <div style={{display:"flex",alignItems:"center",gap:8}}>
            <span className="results-num">{s.results} results</span>
            <button className="detail-btn" onClick={() => setDetailSc
        </div>
    </div>
</div>
    )))
</div>
))

{/* — PROFILE TAB — */}
{tab === "profile" && (
    <div className="tab-content">
        {/* Avatar with glow */}
        <div className="profile-hero">
            <div className="avatar-ring">
                <div className="avatar-inner">
                    <IconUser />
                </div>
            </div>
            <div className="profile-name">{tgUser?.first_name || "GiftTrove"}
            <div className="profile-id">{t.user_id}: {tgUser?.id || "TG_USE
        </div>

        {/* Stars promo - Billy Westor */}
        <a
            className="stars-promo-wrap"
            href="https://t.me/hotontgbot/app?startapp=UQDOUQ2T0pZBQ9d9Df-2"
            target="_blank"
            rel="noreferrer"
        >
            
        </a>

        {/* Sticker cards for Join Channel & Community */}
        <div className="sticker-links">
            <a
                className="sticker-card"

```

```

        href="https://t.me/insidemajek"
        target="_blank"
        rel="noreferrer"
    >
    <img
        src={`https://t.me/i/userpic/160/${joinChannelThumbId}.jpg`}
        alt={t.join_channel}
        className="sticker-img"
        onError={e => {
            e.target.style.display = 'none';
            e.target.nextSibling.style.display = 'flex';
        }}
    />
    <div style={{display:'none', width:60, height:60, alignItems:
        
    <div className="sticker-label">{t.join_channel}</div>
</a>
<a
    className="sticker-card"
    href="https://t.me/+Op7gLVniX9Y1OTRk"
    target="_blank"
    rel="noreferrer"
>
    <img
        src={`https://t.me/i/userpic/160/${communityThumbId}.jpg`}
        alt={t.community}
        className="sticker-img"
        onError={e => {
            e.target.style.display = 'none';
            e.target.nextSibling.style.display = 'flex';
        }}
    />
    <div style={{display:'none', width:60, height:60, alignItems:
        
    <div className="sticker-label">{t.community}</div>
</a>
</div>

{/* Support + links */}
<div className="profile-links">
    <a className="profile-link" href="https://t.me/insidemajek" tar
        <div className="link-icon-wrap">
            
        <span className="link-text">{t.join_channel}</span>
    </a>
</div>

```

```

        <span className="link-arrow"><IconChevronRight /></span>
      </a>
    <div className="divider" />
    <a className="profile-link" href="https://t.me/+Op7gLVniX9Y1OTR"
      <div className="link-icon-wrap">
        
      <span className="link-text">{t.community}</span>
      <span className="link-arrow"><IconChevronRight /></span>
    </a>
    <div className="divider" />
    <a className="profile-link" href="https://t.me/insidemajek?dire
      <div className="link-icon-wrap" style={{background:'rgba(255,
        
      <span className="link-text">{t.support}</span>
      <span className="link-arrow"><IconChevronRight /></span>
    </a>
  </div>

  <div style={{textAlign:"center", marginTop:20, fontSize:11, color
    Built by{" "}
    <a href="https://t.me/insidemajek" style={{color:"var(--tg-blue
  </div>
</div>
  )}
</div>

  { /* TAB BAR */ }
  <div className="tab-bar-wrap">
    <div className="tab-bar">
      {tabs.map(tabItem => (
        <button
          key={tabItem.id}
          className={`tab-item ${tab === tabItem.id ? "active" : ""}`}
          onClick={() => setTab(tabItem.id)}
        >
          <div className="tab-icon-pill">{tabItem.icon}</div>
          <span className="tab-label">{tabItem.label}</span>
        </button>
      ) )}
    </div>
  </div>
</div>
  )}
</>
);

```

